Assignment 2 – CIS I (601.455/655) Fall 2026

October 15, 2026

Scenario

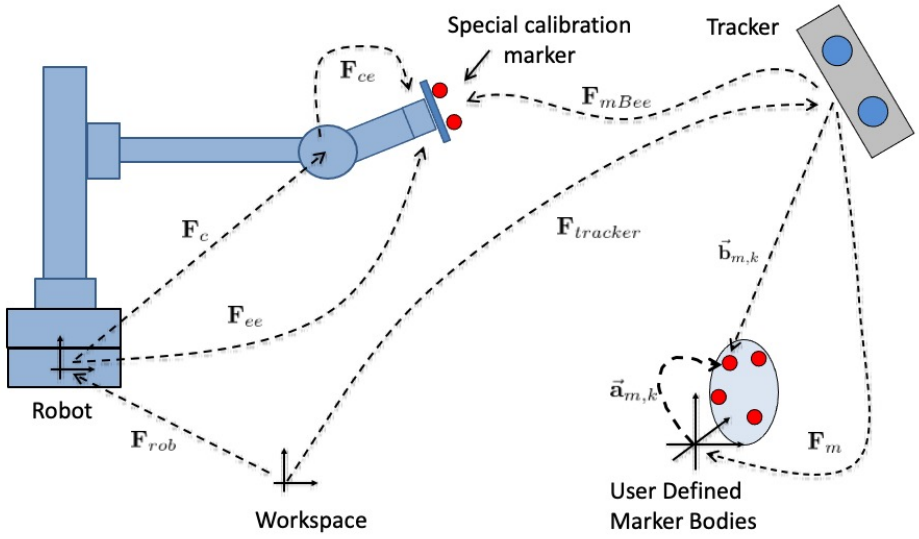


Figure 1: Problem Scenario

This assignment concerns kinematic calibration of a cartesian robot. The goal is to reach a specified accuracy within as short a time as possible. This will be discussed in more detail later. Note that the simulated system will keep a simulated elapsed time variable, t_{elap}, to keep track. This variable will be updated each time the robot is moved or the tracker is told to sample data.

The Tracker

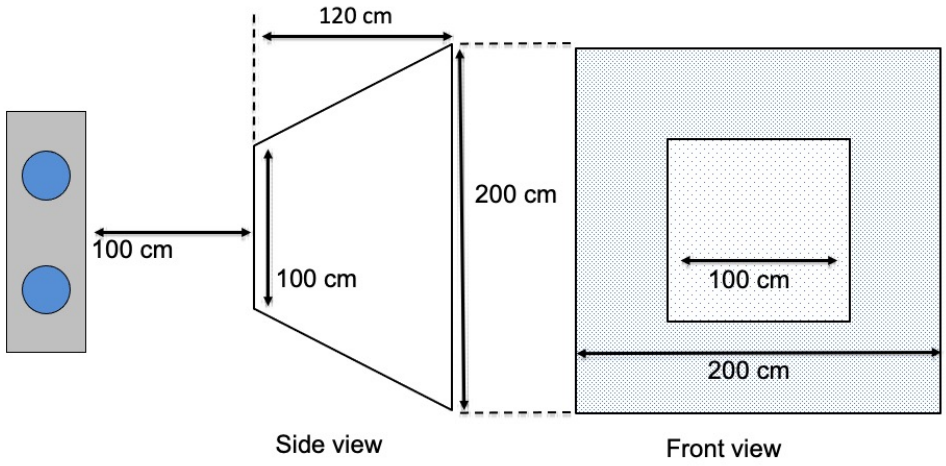


Figure 2: Tracking Camera Workspace

You have access to an optical tracking camera designed to track marker bodies located within a tetrahedral volume shown in Figure 2. As part of this assignment, you should specify an approximate pose $\mathbf{F}_{tracker}$ of the tracking camera relative to the workspace. The actual pose may occasionally "jiggle" with $\mathbf{F}^*_{tracker} = \mathbf{F}_{tracker}\Delta\mathbf{F}_{tracker}$, where $\Delta\mathbf{R} \approx \mathbf{I} + sk(\vec{\alpha}_{tracker})$, $||\vec{\alpha}_{tracker}||_2 \leq \sigma_{Ta}$, and $||\Delta\vec{\mathbf{p}}_{tracker} = \vec{\epsilon}_{tracker}||_2 \leq \sigma_{Tp}$.

A marker body m consists of multiple markers placed at positions $\vec{\mathbf{a}}_{m,k}$ relative to the marker coordinate system. A function

$$mB = DefineMarkerBody(\{\vec{\mathbf{a}}_{m.1}, \cdots, \vec{\mathbf{a}}_{m,N}\})$$

is available to define marker bodies. The actual position $\vec{\mathbf{a}}^*_{m,k}$ of each marker will differ from the specified position by an unknown amount $\Delta\vec{\mathbf{a}}_{m,k}$ with $||\Delta\vec{\mathbf{a}}_{m,k}|| \leq \delta_{markers}$. **Note:** It is possible to improve your estimate of $\vec{\mathbf{a}}_{m,b}$ through a calibration process involving multiple observations of the marker body using the tracking camera and then averaging them. A function

$$PreviousValues = UpdateMarkerBody(mB, \{\vec{\mathbf{a}}_{m.1}, \cdots, \vec{\mathbf{a}}_{m,N}\})$$

is available to modified the user values for the $\vec{\mathbf{a}}_{m,k}$. The simulation system will not modify the $\vec{\mathbf{a}}^*_{m,k}$ values to simulate the tracker observations $\vec{\mathbf{b}}_{m,k}$ of the individual markers.

If an individual marker is located at $\vec{\mathbf{b}}^*$ within the tracking camera's viewing volume, then the value reported by the tracking camera will be $\vec{\mathbf{b}}$, where $\vec{\mathbf{b}}^* = \vec{\mathbf{b}} + \vec{\epsilon}^{trk}_{sys}(\vec{\mathbf{b}}) + \vec{\epsilon}_b$ where $\vec{\epsilon}_b \sim \mathcal{N}(\vec{\mathbf{0}}, \sigma_b^2\mathbf{I}_{3\times3})$, $||\vec{\epsilon}^{trk}_{sys}(\vec{\mathbf{b}})|| \leq \delta^{trk}_{sys}$, and $||\nabla\vec{\epsilon}^{trk}_{sys}|| \leq \omega^{trk}_{sys}$.

A function

$$PlaceMarkerBody(mB, \mathbf{F}_{mb})$$

is available to place a marker body at an approximate pose $\mathbf{F}_{mB}$ relative to the workspace. The actual position of the marker body will be given by

$$\begin{aligned} \mathbf{F}^*_{mB} &= \Delta\mathbf{F}_{mB}\mathbf{F}_{mB} \\ \Delta\mathbf{F}_{mB} &\approx Frame(\mathbf{I} + sk(\vec{\alpha}_{mB}),\ \vec{\epsilon}_{mB}) \end{aligned}$$

where

$$||\vec{\alpha}_{mb}||_2 \leq \delta_{mBa} \qquad ||\vec{\epsilon}_{mb}||_2 \leq \delta_{mBe}$$

A function

$$trk = CreateTracker(\mathbf{F}, \{mB_1, \cdots, mB_N\})$$

Creates a tracker camera, places it approximately at position $\mathbf{F}$ and tells the tracking camera to begin tracking the listed marker bodies. This list can be empty, since there are functions:

$$\begin{aligned} PreviousList &= AddMarkerBodies(trk, \{mB_1, \cdots, mB_N\}) \\ PreviousList &= RemoveMarkerBodies(trk, \{mB_1, \cdots, mB_N\}) \end{aligned}$$

to add and remove marker bodies.

A function

$$\{\mathbf{F}_1, \cdots, \mathbf{F}_N\} = SampleMarkerBodies(trk)$$

will cause the tracking camera to read the markers on all the marker bodies, save the $\vec{\mathbf{b}}_{m,k}$ values with the corresponding marker bodies, and compute and return the values of the corresponding $\{F_1, \cdots, \mathbf{F}_N\}$. Each call to this function requires Δt_{samp} seconds. Two functions are available to retrieve information from the last observation of a tracking camera associated with a marker body mB:

$$\begin{aligned} \mathbf{F}_m &= MostRecentObservedFrame(mB) \\ \{\vec{\mathbf{a}}_{m.1}, \cdots, \vec{\mathbf{a}}_{m,N}\} &= MostRecentObservedMarkers(mB) \end{aligned}$$

Typical values for these parameters are below:

$$\begin{array}{rclrcl} \sigma_{Ta} & = & 0.01 & \sigma_{Tp} & = & 5mm \\ \sigma_b & = & 0.1\,mm & \Delta t_{samp} & = & 0.03\,sec \\ \delta^{trk}_{sys} & \leq & 0.1\,mm & \omega^{trk}_{sys} - & \leq & 7.5 \times 10^{-4} \end{array}$$

Note: For the purposes of this assignment, we will (unrealistically) assume that the camera has no "jiggle". I.e., $\sigma_{Ta} = \sigma_{Tp} = 0$.

The Robot

The robot is located at $\mathbf{F}_{rob}$ in the surgical workspace and consists of a cartesian base and a spherical wrist with a modular tool holder. The nominal kinematic equations[1] are given by:

$$\begin{array}{rcl} \mathbf{F}_{ee}(\vec{\theta}) & = & \mathbf{F}_c(\theta_1, \theta_s, \theta_3)\mathbf{F}_{cw}\mathbf{F}_w(\theta_4, \theta_5, \theta_6)\mathbf{F}_{we} \\ \mathbf{F}_c(\theta_1, \theta_s, \theta_3) & = & \theta_1\vec{\mathbf{d}}_1 + \theta_2\vec{\mathbf{d}}_2 + \theta_3\vec{\mathbf{d}}_3 \\ \mathbf{F}_w(\theta_4, \theta_5, \theta_6) & = & \mathbf{R}(\vec{\mathbf{n}}_4, \theta_4)\mathbf{R}(\vec{\mathbf{n}}_5, \theta_5)\mathbf{R}(\vec{\mathbf{n}}_6, \theta_6) \\ \mathbf{F}_{cw} & = & Frame(\mathbf{I}, [500, 0, 0]^T) \\ \mathbf{F}_{we} & = & Frame(\mathbf{I}, [100, 0, 0]^T) \\ (\vec{\mathbf{d}}_1, \vec{\mathbf{d}}_2, \vec{\mathbf{d}}_3) & = & (\vec{\mathbf{x}}, \vec{\mathbf{y}}, \vec{\mathbf{z}}) \\ (\vec{\mathbf{n}}_1, \vec{\mathbf{n}}_2, \vec{\mathbf{n}}_3) & = & (\vec{\mathbf{x}}, \vec{\mathbf{y}}, \vec{\mathbf{z}}) \\ (0, -300, 0)\,mm & \leq & (\theta_1, \theta_2, \theta_3) \leq (600, 300, 600)\,mm \\ (-0.75\pi, -0.75\pi, -\pi) & \leq & (\theta_4, \theta_5, \theta_6) \leq (0.75\pi, 0.75\pi, \pi) \end{array}$$

For notational convenience, we will also use $\vec{\mathbf{d}}_{k+3} = \vec{\mathbf{n}}_k$. The range of motion is as follows

The actual kinematics are subject to errors. The axes of $\mathbf{F}_c$ are not quite perpendicular, and the linear stages and revolute joints have some distortion and random error. Thus, we have

$$\begin{array}{rcll} \theta^*_k & = & \theta_k + \Delta\theta^{rob}_{k,rnd} + \Delta\theta^{rob}_{k,sys} & \text{where } \Delta\theta^{rob}_{k,rnd} \sim \mathcal{N}(0, \sigma^2_{k,rnd}) \\ \theta_k & = & \theta^*_k - \Delta\theta^{rob}_{k,rnd} - \Delta\theta^{rob}_{k,sys} & \end{array}$$

Although it is small, the distortion is not negligible. However, we know that $||\Delta\theta^{rob}_{k,sys}|| \leq \delta^{rob}_{k,sys}$ and

$$\left| \frac{d}{d\theta_k} \Delta\theta_{k,sys} \right| \leq \omega^{rob}_{k,sys}$$

Note that the $State()$ function discussed below returns the θ_k values. Similarly, we have $\vec{\mathbf{d}}^*_1 = \vec{\mathbf{d}}_1$ and

$$\vec{\mathbf{d}}^*_k = \mathbf{R}(\vec{\phi}_k)\vec{\mathbf{d}}_k \approx \vec{\mathbf{d}}_k + sk(\mathbf{R}(\vec{\phi}_k)\vec{\mathbf{d}}_k) \text{ for } 2 \leq k \leq 6$$

where $||\vec{\phi}_k|| \leq \phi_{max}$. We also have

$$\mathbf{F}^*_{we} = \mathbf{F}_{we}\Delta\mathbf{F}_{ee}(\vec{\eta}_{ee}) \text{ where } ||\vec{\alpha}_{ee}|| \leq 0.01 \text{ and } ||\vec{\epsilon}_{ee}|| \leq 0.5$$

[1] Units are mm and radians

The robot provides the following functions:

$\mathbf{F}$	=	$FwdKins(\vec{\theta})$	returns the value of $\mathbf{F}_{ee}$ corresponding to $\vec{\theta}$
$\vec{\theta}$	=	$InvKins(\mathbf{F})$	returns the value of $\vec{\theta}$ for which $\mathbf{F} = FwdKins(\vec{\theta})$
$(\vec{\theta}, \dot{\vec{\theta}}, \cdots)$	=	$State()$	returns the robot controller's current internal state
$\mathbf{F}$	=	$CommandedPose()$	returns the current commanded value of $\mathbf{F}_{ee}$
Δt	=	$MoveJoints(\vec{\theta})$	moves the robot in time Δt so that $\mathbf{F}_{ee} = FwdKins(\vec{\theta})$
Δt	=	$MoveRobot(\mathbf{F})$	moves the robot in time Δt so that $\mathbf{F}_{ee} = \mathbf{F}$

Note that $MoveRobot(\mathbf{F})$ and $CommandedPose$ are implemented as follows:

$$\begin{aligned} MoveRobot(\mathbf{F}) &= MoveJoints(InvKins(\mathbf{F})) \\ CommandedPose(\vec{\theta}) &= FwdKins(\vec{\theta}) \end{aligned}$$

This information will be important for calibration. Essentially, you will be asked to implement an improved version of the kinematics function to account for the results of calibrating.

The robot has a special homing function.

$$\vec{\theta}_{encoders} = HomeRobot()$$

will place the robot at a "home" position win which $\theta_k^* = 0$ and return the current values read by the encoders (i.e. those returned by $State()$. This may prove useful as part of your calibration process.

Each joint θ_k of the robot has limited travel, speed, and acceleration

$$\theta_k^{min} \le \theta_k \le \theta_k^{max} \qquad \theta_k |\dot{\theta}_k| \le \dot{\theta}_k^{(max)} \qquad |\ddot{\theta}_k| \le \ddot{\theta}_k^{(max)}$$

The time required for the motion will be t_{settle} (s) plus the time required for the slowest joint to move from its current position to the commanded position.

The function

$$Rob = CreateRobot(\mathbf{F}_{rob}, \alpha_{rob}^{max}, \epsilon_{rob}^{max}, \vec{\theta}^{min}, \vec{\theta}^{max}, t_{settle}, \phi_{max}, \sigma_{k,rnd}, |\dot{\vec{\theta}}_{max}|, |\ddot{\vec{\theta}}_{max}|)$$

will create a robot and place it approximately at $\mathbf{F}_{rob}$ in the workspace. The actual position will be

$$\mathbf{F}_{rob}^* = \Delta\mathbf{F}_{rob}\mathbf{F}_{rob} = Frame(Rot(\vec{\alpha}_{rob}), \vec{\epsilon}_{rob})$$

where $||\vec{\alpha}_{rob}|| \le \alpha_{rob}^{max}$ and $||\vec{\epsilon}_{rob}|| \le \epsilon_{rob}^{max}$.

The values for the parameters are

$$
\begin{array}{lcll}
\alpha_{rob}^{max} = 0.05\,radians & & & \\
\epsilon_{rob}^{max} = 5\,mm, & & & \\
\phi_{max} & = & 0.02\,radians & \\
\sigma_{k,rnd} & = & 0.005\,mm & \text{for } 1 \le k \le 3 \\
\sigma_{k,rnd} & = & 0.0001\,radians & \text{for } 4 \le k \le 6 \\
\sigma_{k,sys} & = & 0.01\,mm & \text{for } 1 \le k \le 3 \\
\sigma_{k,sys} & = & 0.001\,radians & \text{for } 4 \le k \le 6 \\
\delta_{sys}^{rob} & = & 0.2\,mm & \text{for } 1 \le k \le 3 \\
\delta_{k,sys}^{rob} & = & 0.01\,radians & \text{for } 4 \le k \le 6 \\
\omega_{k,sys}^{rob} & = & 0.0005 & \text{for } 1 \le k \le 3 \\
\omega_{k,sys}^{rob} & = & 0.00005 & \text{for } 4 \le k \le 6 \\
|\dot{\vec{\theta}}_{max}| & = & 50\,mm/sec & \text{for } 1 \le k \le 3 \\
|\ddot{\vec{\theta}}_{max}| & = & 500\,mm/sec^2 & \text{for } 1 \le k \le 3 \\
|\dot{\vec{\theta}}_{max}| & = & 0.1\,rad/sec & \text{for } 4 \le k \le 6 \\
|\ddot{\vec{\theta}}_{max}| & = & 1\,rad/sec^2 & \text{for } 4 \le k \le 6
\end{array}
$$

For calibration purposes, you can create ONE special marker body $mBee$ that may be attached to the robot's end-effector. You can specify the $\vec{\mathbf{a}}_k^*$ values for this one marker body. The function to do this is:

$$mBee = CreateRobotCalibrationBody(\{\vec{\mathbf{a}}_1^*, \cdots, \vec{\mathbf{a}}_N^*\})$$

where the $\vec{\mathbf{a}}_k^*$ are the actual values. Also, the design of this marker body ensures that its coordinate system exactly corresponds to $\mathbf{F}_{ee}$. The assumption is that it is not unreasonable to have one specially machined calibration body. But you still need to define it. Once this body has been defined for robot *Rob*, it cannot be redefined. Also, the size is restricted, so that

$$||\vec{\mathbf{a}}_k^*||_2 \le 75\,mm$$

Depending on the specific application, it may be desirable to use a smaller size. The specific choice is a design choice. The functions

$$AttachCalibrationMarker(Rob)$$
$$DetachCalibrationMerker(Rob)$$

May be used to attach and detach the marker body to the end effector.

Note: Once it is created, the marker body definition will be saved as an internal variable of the robot object. Any time that the calibration marker body is attached to the robot, it's position relative to the robot base will be updated, so that $\mathbf{F}_{mBee} = \mathbf{F}_{ee}$. Its position in the workspace will be $\mathbf{F}_{rob}\mathbf{F}_{mBee}$.

Problem and a few hints

As mentioned above, This assignment concerns kinematic calibration of a cartesian robot. The goal is to reach a specified accuracy within as short a time as possible. In particular, if

$$\mathbf{F}_{ee}^* = \mathbf{F}_{ee}\Delta\mathbf{F}_{ee} \quad \text{with } \Delta\mathbf{F}_{ee} \approx Frame(\mathbf{I} + sk(\vec{\alpha}_{ee}), \vec{\epsilon}_{ee})$$

Your goal is to produce updated kinematics functions

$$
\begin{aligned}
\mathbf{F}_{ee} &= ImprovedFwdKins(\vec{\theta}) \\
\vec{\theta} &= ImprovedInvKins(\mathbf{F}_{ee})
\end{aligned}
$$

so that we have

$$\begin{aligned}
\Delta\mathbf{F} &= (\mathbf{F}^*_{ee}(\vec{\theta})^{-1} ImprovedFwdKins(\vec{\theta}) \approx Frame(\mathbf{I} + sk(\vec{\alpha}_{FwdK}), \vec{\epsilon}_{FwdK}) \\
||\vec{\alpha}_{FwdK}|| &\leq 10^{-3}\, radians \\
||\vec{\epsilon}|_{FwdK}| &\leq 0.2\, mm
\end{aligned}$$

$$\begin{aligned}
\vec{\theta} &= ImprovedInvKins(\mathbf{F}) \\
\Delta\mathbf{F} &= \mathbf{F}^{-1} FwdKins(\vec{\theta}) \approx Frame(\mathbf{I} + sk(\vec{\alpha}_{iInvK}), \vec{\epsilon}_{Invk}) \\
||\vec{\alpha}_{InvK}|| &\leq 10^{-3}\, radians \\
||\vec{\epsilon}_{InvK}|| &\leq 0.2\, mm
\end{aligned}$$

over the work volume of the robot. To do this, you will need to do the following:

- Define a tracking camera and specify its approximate pose in the workspace.
- Create a robot and specify its approximate pose $\mathbf{F}_{rob}$ in the workspace
- Define marker bodies and specify approximately where they are to be placed in the workspace.
- Define and implement an appropriate workflow for gathering and reducing the data.
- Evaluate your results over the robot work volume. Report accuracy statistics and the time required for calibration and for testing.
- **Note:** If you are unable to reach the desired accuracy, then report the accuracy achieved and how you went about it.

Your development process should also include unit tests for your individual components. You need to describe these, as well, and also report the results.

Note that the robot has the nice property that the first three joints are kinematically decoupled from the last three. You may want to consider this in designing your calibration workflow. Also, note that there are likely to be several ways to do this assignment:

1. Estimate the correct kinematic parameters. To do this:
   (a) Determine how close to each other sample points need to be defined in order to produce a desired degree of accuracy. **Hint:** This will can depend on δ^{trk}_{sys}, ω^{trk}_{sys}, $\delta^{rob}_{k,sys}$, and $\omega_{k,sys}$.
   (b) Move the robot to the home position and then systematically move the robot along/about each individual axis while taking data to recover the parameters for that axis. Return to home position after taking data for each axis.
   (c) Write code to determine the angles between the axes and to compute the transformation between θ_k and θ^*_k. You will probably want some sort of interpolation scheme for this.
   (d) Then write code for the corrected forward and inverse kinematics functions. Note that knowing the actual values of θ_k and ϕ_k will enable you to compute θ^*_k and $\mathbf{F}_{ee}$.
   (e) Your program should be able to report the updated kinematic parameters, and your debugging runs should report how they compare to ground truth. You should also do some "bottom line" testing of a reasonably sampled portion of the workspace

.

2. Map the space and interpolate. Here, you will need to move the robot through the work volume systematically and synthesize the appropriate interpolation functions. You can (and should) take advantage of the kinematic decoupling but need to space your data points (and there will be **many more** this way) close enough in order to guarantee that your interpolation function is sufficiently accurate. Your report should include testing of a fairly dense sampling of random points in the workspace.

3. Some combination of both.

Note that you should include unit testing of each significant element of your calibration program and should explain the reasoning behind your design decisions.

Given the class size, **we will need to limit the execution time of your program to 10 minutes** when we run it against our test configuration, so algorithmic efficiency will be important. Method 1 is likely to require less data and may lead to a more efficient way to test your results. You are **strongly** urged to use this method. Method 2 may require less analysis but is more likely to encounter the runtime limit.

Finally, remember that this is a (somewhat) realistic design exercise. There are multiple choices of method and the choice of individual design parameters affecting your error budget should be considered. The various components of your design add up to produce the desired calibration accuracy.

NOTE: The descriptions of the tracking camera and the robot are typical of what you might find in specification sheets and API documents. Part of your design exercise is determining which elements of these specification sheets are relevant to the problem and deciding how to incorporate them into your design. Remember that your report should explain and justify your design approach and decisions.